LinkSoul-AgentSDK 开发手册

变更历史

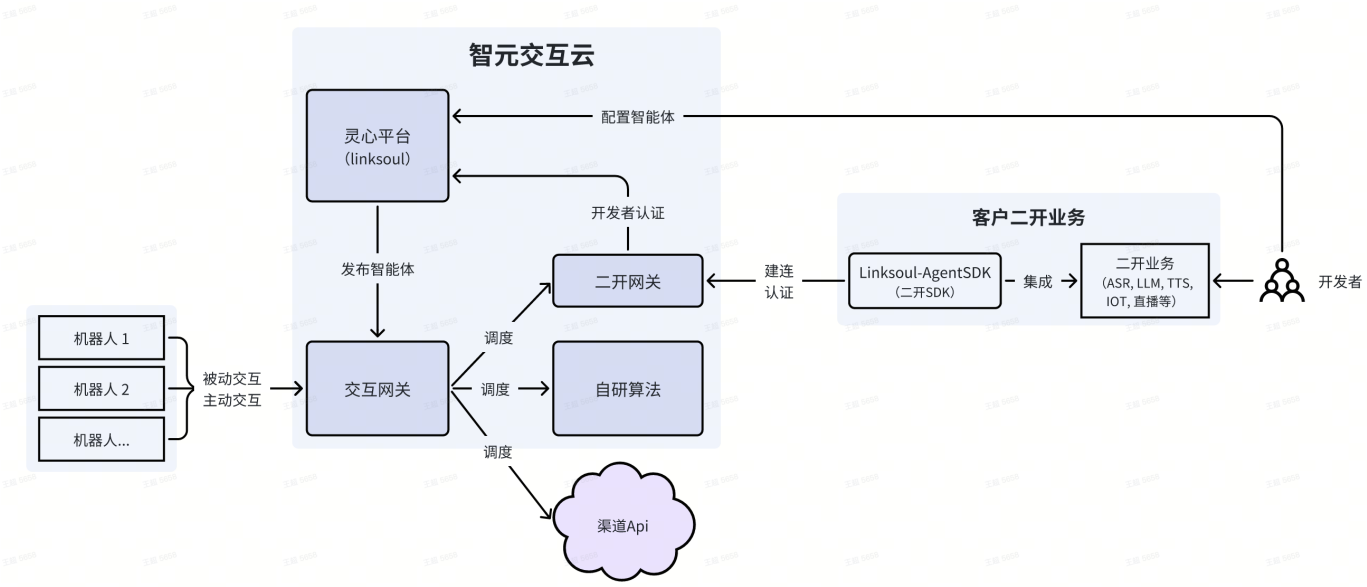
说明：本文档记录SDK各版本变更详情，规范迭代记录，便于二开开发者追溯版本、适配升级，变更类型统一标注（新增/优化/修复/废弃/兼容）。

版本号	发布日期	变更类型	变更内容	影响范围	兼容说明	备注
1.0.0	26/02/10	新增	发布Java语言二开sdk	无	无	支持audio2llm及audio2tts两种二开语义
1.1.0	26/03/06	新增、优化	开放asr及视频流语义	无	授权接口名由AgentSdkAuthCallback变更为AgentAuthCallback，onAuthState回调接口新增appId参数	支持asr2llm、asr2tts、asrVideo2llm、asrVideo2tts、audioVideo2llm及audioVideo2tts

模版使用规范

- 版本号：遵循语义化版本（主版本.次版本.修订版本），如1.0.0、1.2.3，重大变更升级主版本，功能新增升级次版本，Bug修复升级修订版本。
- 变更类型：统一选择「新增/优化/修复/废弃/兼容」，多类型用「、」分隔。
- 影响范围：明确标注变更涉及的SDK版本（Java/Python）、接口、模块，避免开发者遗漏适配。
- 兼容说明：必须明确是否向下兼容，不兼容需标注影响版本及适配方案。

技术架构



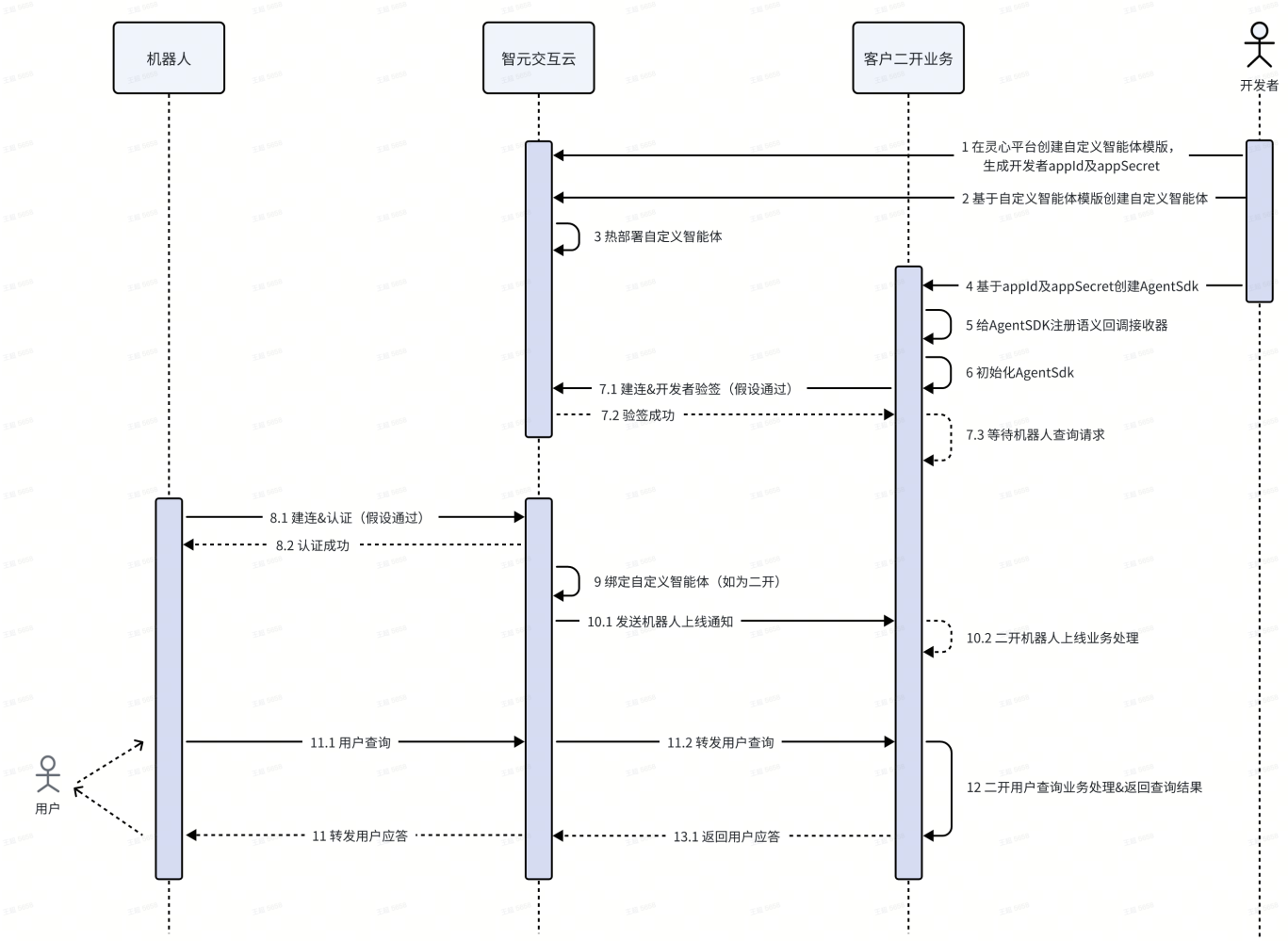
智元云交互平台为二开提供了完整的闭环能力：

- 外部开发者在灵心平台配置并发布自定义智能体，并通过把二开 SDK（Linksoul-AgentSDK）集成到自有业务（如 ASR、LLM、TTS 等场景），经二开网关完成认证与建连。
- 交互网关作为统一出入口，调度自研算法（LLM、VLM、Agent 等）、渠道 API及二开业务，实现与机器人

的主动 / 被动交互。

- 整体架构通过标准化输入输出和统一调度，让二开开发者可快速复用 AI 能力，聚焦业务逻辑，大幅降低集成门槛。

时序图



Java语言SDK

下载地址

linksoul-agentsdk-1.0.0-SNAPSHOT.jar

JDK依赖

为保障代码兼容性、安全性及性能优化，建议采用 JDK 17 及以上版本进行 Java 开发（优先选择长期支持版 LTS）。

第三方开源库依赖清单

依赖组ID (groupId)	依赖项ID (artifactId)	版本号 (version)	说明
org.slf4j	slf4j-api	2.0.16	SLF4J 日志门面 API
org.projectlombok	lombok	1.18.36	简化 Java 代码的工具库
ch.qos.logback	logback-classic	1.5.13	Logback 日志实现
com.alibaba.fastjson2	fastjson2	2.0.53	阿里巴巴 JSON 解析库
io.netty	netty-all	4.1.116.Final	Netty 网络编程框架
org.apache.commons	commons-lang3	3.12.0	Apache 通用语言工具类
io.jsonwebtoken	jjwt-api	0.11.5	JWT 令牌处理 API
io.jsonwebtoken	jjwt-impl	0.11.5	JWT 令牌处理实现类
io.jsonwebtoken	jjwt-jackson	0.11.5	JWT + Jackson 序列化支持

Maven 依赖配置（pom.xml）

将以下内容粘贴到 pom.xml 的 标签内：

```
<dependency>
    <groupId>org.slf4j</groupId>
    <artifactId>slf4j-api</artifactId>
    <version>2.0.16</version>
</dependency>
<dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <version>1.18.36</version>
</dependency>
<dependency>
    <groupId>ch.qos.logback</groupId>
    <artifactId>logback-classic</artifactId>
    <version>1.5.13</version>
</dependency>
<dependency>
    <groupId>com.alibaba.fastjson2</groupId>
    <artifactId>fastjson2</artifactId>
    <version>2.0.53</version>
</dependency>
<dependency>
    <groupId>io.netty</groupId>
    <artifactId>netty-all</artifactId>
    <version>4.1.116.Final</version>
</dependency>
<dependency>
    <groupId>org.apache.commons</groupId>
    <artifactId>commons-lang3</artifactId>
    <version>3.12.0</version>
</dependency>
```

```
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-api</artifactId>
    <version>0.11.5</version>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-impl</artifactId>
    <version>0.11.5</version>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-jackson</artifactId>
    <version>0.11.5</version>
</dependency>
```

gradle依赖

Gradle 依赖配置（Groovy 格式，build.gradle）

```
implementation 'org.slf4j:slf4j-api:2.0.16'
implementation 'org.projectlombok:lombok:1.18.36'
implementation 'org.projectlombok:lombok:1.18.36'
implementation 'ch.qos.logback:logback-classic:1.5.13'
implementation 'com.alibaba.fastjson2:fastjson2:2.0.53'
implementation 'io.netty:netty-all:4.1.116.Final'
implementation 'org.apache.commons:commons-lang3:3.12.0'
implementation 'io.jsonwebtoken:jjwt-api:0.11.5'
implementation 'io.jsonwebtoken:jjwt-impl:0.11.5'
implementation 'io.jsonwebtoken:jjwt-jackson:0.11.5'
```

二开语义

接口名称	接口类型	接口描述	AgentSDK Java版 请求（输入回调）	AgentSDK Java版 应答（输出响应）	开放时间
Audio2Llm	三段式	流式输入音频，流式输出LLM文本	Audio2LlmCallback	Audio2LlmResponse	26/02/10
Audio2Tts	三段式	流式输入音频，流式输出TTS音频	Audio2TtsCallback	Audio2TtsResponse	26/02/10
Asr2Llm	三段式	输入ASR文本，流式输出LLM文本	Asr2LlmCallback	Asr2LlmResponse	26/03/12
Asr2Tts	三段式	输入ASR文本，流式输出TTS音频	Asr2TtsCallback	Asr2TtsResponse	26/03/12
AsrVideo2Tts	三段式	输入ASR文本及h264原始视频帧，流式输出TTS音频	AsrVideo2TtsCallback	AsrVideo2TtsResponse	26/03/12
AsrVideo2Vlm	三段式	输入ASR文本及h264原始视频帧，流式输出VLM文本	AsrVideo2VlmCallback	AsrVideo2VlmResponse	26/03/12
AudioVideo2Tts	三段式	流式输入音频及h264原始视频帧，流式输出TTS音频	AudioVideo2TtsCallback	AudioVideo2TtsResponse	26/03/12
AudioVideo2Vlm	三段式	流式输入音频及h264原始视频帧，流式输出VLM文本	AudioVideo2VlmCallback	AudioVideo2VlmResponse	26/03/12

开发步骤

第一步：创建二开SDK实例

开发者appId及appSecret需从 灵心平台 自定义智能体模版获取。

```
String url = "wss://agentsdk.test.agibot.com/v1/agent";
String appId = "your appId";
String appSecret = "your appSecret";
AgentSdk agentSdk = AgentSdk.create(url, appId, appSecret);
```

第二步 注册开发者验签状态回调接口

```
agentSdk.registerAuth(new AgentAuthCallback() {

    /**
    * @param appId       灵心平台自定义智能体模版的appId
    * @param code        code 0 成功，非0表示认证失败
    * @param msg         认证消息
    */
    @Override
    public void onAuthState(String appId, int code, String msg) {
        //
    }

});
```

第三步：注册被动交互接口

如下语义接口可按需使用。

1 注册Audio2Llm语义回调接口

属于三段式接口，流式输入音频，流式输出ASR文本，流式输出LLM文本。

```
agentSdk.registerAudio2Llm(new Audio2LlmCallback(agentSdk) {

    /**
     * @param agentId     智能体Id（如AGENT_0000001）
     * @param eventId     会话id，一个查询一个唯一Id（如 event_r4aT53w0zKyMGJ4c3dgOS）
     * @param flag        0 VAD开始，1中间音频，2 VAD结束
     * @param buf         音频字节数组（当flag为1时有效）
     * @param response    应答，二开开发者通过该对象接口返回asr识别结果及llm文本结果
     */
    @Override
    public void onRequest(
            String agentId, String eventId, int flag,
            byte[] buf, Audio2LlmResponse response) {

        // TODO 流式音频请求处理实现（做asr识别，基于识别结果做语义理解，基于语义理解结果做RAG、工具调用等，基于RAG或工具调用结果做大模型总结），并返回Asr及Llm结果

    }
```

```
});
```

2 注册Audio2Tts语义回调接口

属于三段式接口，流式输入音频，流式输出ASR文本、流式输出LLM文本，流式输出TTS音频。

```
agentSdk.registerAudio2Tts(new Audio2TtsCallback(agentSdk) {

    /**
     * @param agentId     智能体Id（如AGENT_0000001）
     * @param eventId     会话id，一个查询一个唯一Id（如 event_r4aT53w0zKyMGJ4c3dgOS）
     * @param flag        0 VAD开始，1中间音频，2 VAD结束
     * @param buf         音频字节数组（当flag为1时有效）
     * @param response    应答，二开开发者通过该对象接口返回asr识别结果、llm文本结果及tts音频结果
     */
    @Override
    public void onRequest(
            String agentId, String eventId, int flag,
            byte[] buf, Audio2TtsResponse response) {

        // TODO 流式音频请求处理实现（做asr识别，基于识别结果做语义理解，基于语义理解结果做RAG、工具调用等，基于RAG或工具调用结果做大模型总结，基于大模型总结结果做TTS合成），并返回Asr、Llm及Tts结果

    }
});
```

3 注册Asr2Llm回调接口

属于三段式接口，输入asr识别结果的final结果文本，流式输出LLM文本。

```
agentSdk.registerAsr2Llm(new Asr2LlmCallback(agentSdk) {

    /**
    * @param agentId     智能体Id（如AGENT_0000001）
    * @param eventId     会话id，一个查询一个唯一Id（如 event_r4aT53w0zKyMGJ4c3dgOS）
    * @param text        asr识别final结果
    * @param response    应答，二开开发者通过该对象接口返回asr识别结果及llm文本结果
    */
    @Override
    public void onRequest(
        String agentId, String eventId,
        String text, Asr2LlmResponse response) {

        // TODO 文本请求处理实现（基于文本请求做语义理解，基于语义理解结果做RAG、工具调用等，基于RAG或工具调用结果做大模型总结），并返回Llm结果

    }
});
```

4 注册Asr2Tts回调接口

属于三段式接口，输入asr识别结果的final结果文本，流式输出LLM文本及流式输出TTS音频。

```
agentSdk.registerAsr2Tts(new Asr2TtsCallback(agentSdk){

    /**
    * @param agentId    智能体Id（如AGENT_0000001）
    * @param eventId    会话id，一个查询一个唯一Id（如 event_r4aT53w0zKyMGJ4c3dgOS）
    * @param text       asr识别final结果
    * @param response   应答，二开开发者通过该对象来返回asr识别结果、llm文本结果及tts音频结果
    */
    @Override
    public void onRequest(
        String agentId, String eventId,
        String text, Asr2TtsResponse response) {

        // TODO 文本请求处理实现（基于文本请求做语义理解，基于语义理解结果做RAG、工具调用等，基于RAG或工具调用结果做大模型总结，基于大模型总结结果做TTS合成），并返回Llm及Tts结果

    }
});
```

5 注册asrVideo2vlm回调接口

属于三段式接口，输入asr识别结果的final结果文本及视频图片帧，流式输出VLM文本。

```
agentSdk.registerAsrVideo2Vlm(new AsrVideo2VlmCallback(agentSdk){

    /**
    * @param agentId    智能体Id（如AGENT_0000001）
    * @param eventId    会话id，一个查询一个唯一Id（如 event_r4aT53w0zKyMGJ4c3dgOS）
    * @param flag       3 final asr文本，4 h264视频帧
    * @param text       asr识别final结果（当flag为3时有效）
    * @param buf        视频帧图片（当flag为4时有效）
    * @param isKeyFrame 是否为关键帧（当flag为4时有效）
    * @param frameTime  帧采集时间戳（当flag为4时有效）
    * @param response   应答，二开开发者通过该对象接口返回vlm文本结果
    */
    @Override
    public void onRequest(
        String agentId, String eventId, int flag,
        String text, byte[] buf, boolean isKeyFrame, long frameTime, AsrVideo2VlmResponse response) {

        // TODO 文本及h264视频流请求处理实现（基于文本做语义理解，基于图像做人脸识别、vlm场景理解等；基于语义理解结果做RAG、工具调用等；基于RAG或工具调用结果做大模型总结），并返回大模型总结结果

    }
});
```

6 注册asrVideo2tts回调接口(3/12开放)

属于三段式接口，输入asr识别的final结果文本及视频图片帧，流式输出VLM文本及流式输出TTS音频。

```
agentSdk.registerAsrVideo2Tts(new AsrVideo2TtsCallback(agentSdk){

    /**
    * @param agentId    智能体Id（如AGENT_0000001）
    * @param eventId    会话id，一个查询一个唯一Id（如 event_r4aT53w0zKyMGJ4c3dgOS）
    * @param flag       3 final asr文本，4 h264视频帧
    * @param text       asr识别final结果（当flag为3时有效）
    * @param buf        h264视频帧（当flag为4时有效）
    * @param isKeyFrame 是否为关键帧（当flag为4时有效）
    * @param frameTime  帧采集时间戳（当flag为4时有效）
    * @param response   应答，二开开发者通过该对象接口返回vlm文本结果及tts音频结果
    */
    @Override
    public void onRequest(
        String agentId, String eventId, int flag,
        String text, byte[] buf, boolean isKeyFrame, long frameTime, AsrVideo2TtsResponse
response) {

        // TODO 文本及h264视频流请求处理实现（基于文本做语义理解，基于图像做人脸识别、vlm场景理解等；基
于语义理解结果做RAG、工具调用等；基于RAG或工具调用结果做大模型总结；基于大模型总结结果做TTS合成），并返回
大模型总结结果及Tts结果

    }
});
```

7 注册audioVideo2vlm回调接口

属于三段式接口，输入流式音频及视频图片帧，流式输出ASR文本，流式输出VLM文本。

```
agentSdk.registerAudioVideo2Vlm(new AudioVideo2VlmCallback(agentSdk){

    /**
    * @param agentId    智能体Id（如AGENT_0000001）
    * @param eventId    会话id，一个查询一个唯一Id（如 event_r4aT53w0zKyMGJ4c3dgOS）
    * @param flag       0 vad开始，1 音频，2 vad结束，4 h264视频帧，5 jpg图片
    * @param buf        音频字节数组（当flag为1时有效），h264视频帧（当flag为4时有效），jpg图片（当
flag为5时有效）
    * @param isKeyFrame 是否为关键帧（当flag为4时有效）
    * @param frameTime  帧采集时间戳（当flag为4时有效）
    * @param response   应答，二开开发者通过该对象接口返回asr识别结果、vlm文本结果及tts音频结果
    */
    @Override
    public void onRequest(
        String agentId, String eventId, int flag
        byte[] buf, boolean isKeyFrame, long frameTime, AudioVideo2VlmResponse response) {

        // TODO 流式音频及h264视频流请求处理实现（做asr识别，基于识别结果做语义理解，基于图像做人脸识
别、vlm场景理解等；基于语义理解结果做RAG、工具调用等；基于RAG或工具调用结果做大模型总结），并返回asr及大
模型总结结果
```

```
    }
});
```

8 注册audioVideo2tts语义回调接口

属于三段式接口，流式输入音频及视频帧图片，流式输出ASR文本、流式输出VLM文本及流式输出TTS音频。

```
agentSdk.registerAudioVideo2Tts(new AudioVideo2TtsCallback(agentSdk) {

    /**
    * @param agentId    智能体Id（如AGENT_0000001）
    * @param eventId    会话id，一个查询一个唯一Id（如 event_r4aT53w0zKyMGJ4c3dgOS）
    * @param flag       0 vad开始，1 音频，2 vad结束，4 h264视频帧，5 jpg图片
    * @param buf        音频字节数组（当flag为1时有效），h264视频帧（当flag为4时有效），jpg图片（当flag为5时有效）
    * @param isKeyFrame 是否为关键帧（当flag为4时有效）
    * @param frameTime  帧采集时间戳（当flag为4时有效）
    * @param response   应答，二开开发者通过该对象接口返回asr识别结果、vlm文本结果及tts音频结果
    */
    @Override
    public void onRequest(
        String agentId, String eventId,
        int flag, byte[] buf, boolean isKeyFrame, long frameTime, AudioVideo2TtsResponse response) {

        // TODO 流式音频及h264视频流请求处理实现（做asr识别，基于识别结果做语义理解，基于图像做人脸识别、vlm场景理解等；基于语义理解结果做RAG、工具调用等；基于RAG或工具调用结果做大模型总结；基于大模型总结结果做Tts合成），并返回asr、大模型总结及tts结果

    }
});
```

第四步：初始化SDK

初始化成功，验签回调接口会返回code为0的状态码，表示开发者验签成功。
当机器人上线后会回调机器人上线回调接口，只有机器人上线，才能进行二开业务路由。

```
sdk.initialize();
```

非0表示验签失败，常见验签失败错误码参考如下认证错误码表格。

验签失败错误码：

错误码	错误描述	解决建议
1000	缺少 JWT Token，请在请求头携带 AGIBOT_AUTHORIZATION: Bearer	在http头中携带key为AGIBOT_AUTHORIZATION，value为Bearer与token拼接的字符串
1001	Token 已过期，请重新获取	重连
1002	Token 格式错误	jwt签名为3段，用2个.号连接
1003	Token 签名验证失败（非法Token）	秘钥不对或appId与秘钥不成对
1004	Token 为空或载荷非法	无token或其他原因
1005		

开发示例

1 Audio2Llm 接口开发示例

本示例基于 LinkSoul-AgentSdk 实现音频转语义（Audio2Llm）能力，核心流程为：初始化 AgentSdk 实例并完成验签回调注册，
通过注册 Audio2LlmCallback 监听音频交互事件（VAD 开始 / 中间音频 / VAD 结束），在回调中对接外部 ASR 服务完成音频转文字，
再基于 ASR 结果调用 LLM 服务生成语义回复，最后通过 Audio2LlmResponse 接口流式返回 ASR 中间 / 最终结果、LLM 分意图流式结果及结束状态，异常场景可返回错误码与描述信息。

```
import com.agibot.aiem.sdk.AgentSdk;
import com.agibot.aiem.sdk.AgentMeta;
import com.agibot.aiem.sdk.AgentAuthCallback;
import com.agibot.aiem.sdk.passive.Audio2LlmCallback;
import com.agibot.aiem.sdk.passive.Audio2LlmResponse;

public class Audio2LlmExample {

    public static void main(String[] args) {

        String url = "wss://agentsdk.test.agibot.com/v1/agentsdk";
        String appId = "您通过灵心平台创建的二开智能体模版获取的appId";
        String appSecret = "您通过灵心平台创建的二开智能体模版获取的appSecret";

        // 第一步：创建sdk实例
        AgentSdk agentSdk = AgentSdk.create(url, appId, appSecret);

        // 第二步：注册验签回调接口
        agentSdk.registerAuth(new AgentAuthCallback() {
            @Override
            public void onAuthState(String appId, int code, String msg) {
                System.out.println("onAuthState => appId: " + appId + "code: " + code + ",
msg: " + msg);
            }
        });
```

```
        // 第三步：注册语义回调接口
        // 注册audio2llm语义接口
        agentSdk.registerAudio2Llm(new Audio2LlmCallback(agentSdk) {

            // TODO 因该实例为单例，非线程安全的资源对象可基于eventId做隔离，并在用完后及时销毁

            @Override
            public void onRobotOnline(AgentMeta agentMeta) {
                // 可选重载
                System.out.println("onRobotOnline => agentMeta " +
agentMeta.toJSONString());
            }

            @Override
            public void onRobotOffline() {
                // 可选重载
            }

            /**
             * @param agentId    智能体Id（如AGENT_0000001）
             * @param eventId    会话id，一个查询一个唯一Id（如 event_r4aT53w0zKyMGJ4c3dgOS）
             * @param flag       0 VAD开始，1中间音频，2 VAD结束
             * @param buf        音频字节数组（当flag为1时有效）
             * @param response   应答，二开开发者通过该对象接口返回asr识别结果及llm文本结果
             */
            @Override
            public void onRequest(
                    String agentId, String eventId, int flag,
                    byte[] buf, Audio2LlmResponse response) {

                // 可选获取自定义智能体默认元数据及自定义元数据
                AgentMeta agentMeta = agentSdk.getAgentMeta(agentId);
                // 智能体默认配置可直接通过函数名获取
                String defWakeupWord = agentMeta.getWakeupWord();
                String defCity = agentMeta.getCity();
                String defRobotName = agentMeta.getRobotName();
                String defBuzzWord = agentMeta.getBuzzWord();
                // 智能体自定义配置可通过参数key获取
                String yourParamKey = agentMeta.getString("您的自定义参数key");


                // TODO 调用ASR逻辑
                if (flag == 0) {
                    // 如初始化asr的客户端
                } else if (flag == 1) {
                    // 给asr传音频
                } else if (flag == 2) {
                    //
                }
                String asrMiddle = "今天"; // 假设这是一个asr的中间识别结果。
                String asrFinal = "今天天气咋样"; // 假设这是一个asr的中间识别结果。
```

```
                // 当ASR服务返回流式中间结果，则通过response应答中间结果
                response.onAsrMiddle(eventId, asrMiddle);
                // 当ASR服务返回最终结果，则通过response应答最终结果
                response.onAsrFinal(eventId, asrFinal);

                // TODO 拿asrFinal（以"今天天气怎么样"为例）做语义理解（NLU），基于语义理解结果去做知识库查询、工具调用等，再拿知识库查询结果或工具调用结果调用LLM做总结

                // LLM结果可能是多意图，需要给每个意图生成一个唯一的itemId，假设如下
                String itemId = "itemId_000001";
                String textDelta = "假设这就是LLM流式生成的结果";
                // 当LLM服务生成（某个意图）流式文本delta，则通过response应答（某个意图）流式文本delta
                response.onLlmItemDelta(eventId, itemId, textDelta);
                // 当LLM服务生成（某个意图）结束，则通过response应答（某个意图）结束状态
                response.onLlmItemDone(eventId, itemId);
                // 当LLM服务生成（所有意图）结束，则通过response应答（所有意图）结束状态
                response.onLlmDone(eventId);

                // 当异常时，通过response返回异常信息
                /*
                int errorCode = 1010;
                String errorMsg = "假设调用asr出错了";
                response.onError(eventId, errorCode, errorMsg);
                */
            }
        });

        // 第四步：初始化
        agentSdk.initialize();
    }

}
```

2 Audio2Tts 接口开发示例

本示例基于 LinkSoul-AgentSdk 实现音频转语义再转语音的三段式流式交互：先替换代码中url、appId、appSecret为生产环境值，
再在onRequest方法中对接实际的 ASR（音频转文本）、LLM（语义处理）、TTS（文本转音频）服务，按flag（VAD 状态）处理流式音频，
通过response系列方法回传 ASR/LLM/TTS 的流式 / 最终结果（异常时调用onError反馈）；部署时需隔离eventId维度的非线程安全资源，
启动程序后 Sdk 自动完成鉴权初始化，监听并处理智能体端的音频请求。

```
import com.agibot.aiem.sdk.AgentSdk;
import com.agibot.aiem.sdk.AgentMeta;
import com.agibot.aiem.sdk.AgentAuthCallback;
import com.agibot.aiem.sdk.passive.Audio2LlmCallback;
import com.agibot.aiem.sdk.passive.Audio2LlmResponse;
```

```
public class Audio2TtsMainApp {

    public static void main(String[] args) {

        String url = "wss://agentsdk.test.agibot.com/v1/agentsdk";
        String appId = "您通过灵心平台创建的二开智能体模版获取的appId";
        String appSecret = "您通过灵心平台创建的二开智能体模版获取的appSecret";

        // 第一步：创建sdk实例
        AgentSdk agentSdk = AgentSdk.create(url, appId, appSecret);

        // 第二步：注册验签回调接口
        agentSdk.registerAuth(new AgentAuthCallback() {
            @Override
            public void onAuthState(String appId, int code, String msg) {
                System.out.println("onAuthState => appId: " + appId + "code: " + code + ",
msg: " + msg);
            }
        });

        // 第三步：注册语义回调接口
        // 注册audio2llm语义接口
        agentSdk.registerAudio2Llm(new Audio2TtsCallback(agentSdk) {

            // TODO 因该实例为单例，非线程安全的资源对象可基于eventId做隔离，并在用完后及时销毁

            @Override
            public void onRobotOnline(AgentMeta agentMeta) {
                // 可选重载
                System.out.println("onRobotOnline => agentMeta " +
agentMeta.toJSONString());
            }

            @Override
            public void onRobotOffline() {
                // 可选重载
            }

            /**
             * @param agentId    智能体Id（如AGENT_0000001）
             * @param eventId    会话id，一个查询一个唯一Id（如 event_r4aT53w0zKyMGJ4c3dgOS）
             * @param itemId     意图id，一个意图一个唯一Id（如 item_r4aT53w0zKyMGJ4c3dgOS）
             * @param flag       0 VAD开始，1中间音频，2 VAD结束
             * @param buf        音频字节数组（当flag为1时有效）
             * @param response   应答，二开开发者通过该对象接口返回asr识别结果及llm文本结果
             */
            @Override
            public void onRequest(
                    String agentId, String eventId, String itemId, int flag,
                    byte[] buf, Audio2LlmResponse response) {

                // 可选获取自定义智能体默认元数据及自定义元数据
```

```
                AgentMeta agentMeta = agentSdk.getAgentMeta(agentId);
                // 智能体默认配置可直接通过函数名获取
                String defWakeupWord = agentMeta.getWakeupWord();
                String defCity = agentMeta.getCity();
                String defRobotName = agentMeta.getRobotName();
                String defBuzzWord = agentMeta.getBuzzWord();
                // 智能体自定义配置可通过参数key获取
                String yourParamKey = agentMeta.getString("您的自定义参数key");


                // TODO 调用ASR逻辑
                if (flag == 0) {
                    // 如初始化asr的客户端
                } else if (flag == 1) {
                    // 给asr传音频
                } else if (flag == 2) {
                    //
                }
                String asrMiddle = "今天"; // 假设这是一个asr的中间识别结果。
                String asrFinal = "今天天气咋样"; // 假设这是一个asr的中间识别结果。

                // 当ASR服务返回流式中间结果，则通过response应答中间结果
                response.onAsrMiddle(eventId, asrMiddle);
                // 当ASR服务返回最终结果，则通过response应答最终结果
                response.onAsrFinal(eventId, asrFinal);

                // TODO 拿asrFinal（以"今天天气怎么样"为例）做语义理解（NLU），基于语义理解结果做RAG知识查询、工具调用等，再拿知识库查询或工具调用结果调用LLM做总结

                // LLM结果可能是多意图，需要给每个意图生成一个唯一的itemId，假设如下
                String itemId = "itemId_000001";
                String textDelta = "假设这就是LLM流式生成的结果";
                // 当LLM服务生成（某个意图）流式文本delta，则通过response应答（某个意图）流式文本delta
                response.onLlmItemDelta(eventId, itemId, textDelta);
                // 当LLM服务生成（某个意图）结束，则通过response应答（某个意图）结束状态
                response.onLlmItemDone(eventId, itemId);
                // 当LLM服务生成（所有意图）结束，则通过response应答（所有意图）结束状态
                response.onLlmDone(eventId);

                // TODO 调用TTS逻辑
                byte[] audio = new byte[1024]; // 假设这是tts服务合成的音频audio

                // 当TTS服务生成itemId的流式tts音频delta，则通过response应答流式tts音频delta
                response.onTtsItemDelta(eventId, itemId, audio);
                // 当TTS服务生成itemId的流式tts音频结束，则通过response应答流式tts音频结束状态
                response.onTtsItemDone(eventId, itemId);
                // 当TTS服务生成全部结束，则通过response应答全乎结束状态
                response.onTtsDone(eventId);


                // 当异常时，通过response返回异常信息
```

```
                /*
                int errorCode = 1010;
                String errorMsg = "假设调用asr出错了";
                response.onError(eventId, errorCode, errorMsg);
                */
            }
        });

        // 第四步：初始化
        agentSdk.initialize();
    }

}
```

3 Asr2Llm 接口开发示例

本示例基于 LinkSoul-AgentSdk 实现文本语义处理能力：先替换代码中url、appId、appSecret为生产环境值，再在onRequest方法中对接实际的 NLU（语义理解）、RAG（知识库查询）、LLM（文本生成）服务，针对传入的 ASR
最终文本完成语义处理和多意图流式文本生成，通过response系列方法回传 LLM 的流式 / 最终结果（异常时调用onError反馈）；
部署时需隔离eventId维度的非线程安全资源，启动程序后 Sdk 自动完成鉴权初始化，监听并处理智能体端的文本语义请求。

```
import com.agibot.aiem.sdk.AgentMeta;
import com.agibot.aiem.sdk.AgentSdk;
import com.agibot.aiem.sdk.passive.*;

public class Asr2LlmExample {

    public static void main(String[] args) {

        String url = "wss://agentsdk.test.agibot.com/v1/agentsdk";
        String appId = "您通过灵心平台创建的二开智能体模版获取的appId";
        String appSecret = "您通过灵心平台创建的二开智能体模版获取的appSecret";

        // 第一步：创建sdk实例
        AgentSdk agentSdk = AgentSdk.create(url, appId, appSecret);

        // 第二步：注册验签回调接口
        agentSdk.registerAuth(new AgentAuthCallback() {
            @Override
            public void onAuthState(String appId, int code, String msg) {
                System.out.println("onAuthState => appId: " + appId + "code: " + code + ",
msg: " + msg);
            }
        });

        // 第三步：注册语义回调接口
        // 注册asr2llm语义接口
        agentSdk.registerAsr2Llm(new Asr2LlmCallback(agentSdk) {
```

```
            // TODO 因该实例为单例，非线程安全的资源对象可基于eventId做隔离，并在用完后及时销毁

            @Override
            public void onRobotOnline(String agentId, AgentMeta agentMeta) {
                // 可选重载
                System.out.println("onRobotOnline => agentMeta " + agentMeta.toString());
            }

            @Override
            public void onRobotOffline(String agentId) {

            }

            /**
             * @param agentId    智能体Id（如AGENT_0000001）
             * @param eventId    会话id，一个查询一个唯一Id（如 event_r4aT53w0zKyMGJ4c3dgOS）
             * @param text       查询文本，一般为final asr
             * @param response   应答，二开开发者通过该对象接口返回asr识别结果及llm文本结果
             */
            @Override
            public void onRequest(
                    String agentId, String eventId, String text,
                    Asr2LlmResponse response) {

                // 可选获取自定义智能体默认元数据及自定义元数据
                AgentMeta agentMeta = agentSdk.getAgentMeta(agentId);
                // 智能体默认配置可直接通过函数名获取
                String defWakeupWord = agentMeta.getWakeupWord();
                String defCity = agentMeta.getCity();
                String defRobotName = agentMeta.getRobotName();
                String defBuzzWord = agentMeta.getBuzzWord();
                // 智能体自定义配置可通过参数key获取
                String yourParamKey = agentMeta.getString("您的自定义参数key");

                // TODO 拿text（如"今天天气怎么样"为例）做语义理解（NLU），基于语义理解结果去做知识库查询、工具调用等，再拿知识库查询结果或工具调用等结果调用LLM做总结

                // LLM结果可能是多意图，需要给每个意图生成一个唯一的itemId，假设如下
                String itemId = "itemId_000001";
                String textDelta = "假设这就是LLM流式生成的结果";
                // 当LLM服务生成（某个意图）流式文本delta，则通过response应答（某个意图）流式文本delta
                response.onLlmItemDelta(eventId, itemId, textDelta);
                // 当LLM服务生成（某个意图）结束，则通过response应答（某个意图）结束状态
                response.onLlmItemDone(eventId, itemId);
                // 当LLM服务生成（所有意图）结束，则通过response应答（所有意图）结束状态
                response.onLlmDone(eventId);

                // 当异常时，通过response返回异常信息
                /*
                int errorCode = 1010;
```

```
                String errorMsg = "假设调用asr出错了";
                response.onError(eventId, errorCode, errorMsg);
                */
            }
        });

        // 第四步：初始化
        agentSdk.initialize();
    }

}
```

4 Asr2Tts 接口开发示例

本示例基于 LinkSoul-AgentSdk 实现文本转语音的端到端处理能力：先替换代码中url、appId、appSecret为生产环境值，
再在onRequest方法中对接实际的 NLU（语义理解）、RAG（知识库查询）、LLM（文本生成）、TTS（语音合成）服务，
针对传入的 ASR 最终文本完成语义处理、多意图流式文本生成及语音合成，通过response系列方法回传 LLM 文本和 TTS 音频的流式 / 最终结果（异常时调用onError反馈）；
部署时需隔离eventId维度的非线程安全资源，启动程序后 Sdk 自动完成鉴权初始化，监听并处理智能体端的文本转语音请求。

```
import com.agibot.aiem.sdk.AgentMeta;
import com.agibot.aiem.sdk.AgentSdk;
import com.agibot.aiem.sdk.passive.*;

public class Asr2TtsExample {

    public static void main(String[] args) {

        String url = "wss://agentsdk.test.agibot.com/v1/agentsdk";
        String appId = "您通过灵心平台创建的二开智能体模版获取的appId";
        String appSecret = "您通过灵心平台创建的二开智能体模版获取的appSecret";

        // 第一步：创建sdk实例
        AgentSdk agentSdk = AgentSdk.create(url, appId, appSecret);

        // 第二步：注册验签回调接口
        agentSdk.registerAuth(new AgentAuthCallback() {
            @Override
            public void onAuthState(String appId, int code, String msg) {
                System.out.println("onAuthState => appId: " + appId + "code: " + code + ",
msg: " + msg);
            }
        });

        // 第三步：注册语义回调接口
        // 注册audio2llm语义接口
        agentSdk.registerAsr2Tts(new Asr2TtsCallback(agentSdk) {
```

```
            // TODO 因该实例为单例，非线程安全的资源对象可基于eventId做隔离，并在用完后及时销毁

            @Override
            public void onRobotOnline(String agentId, AgentMeta agentMeta) {
                // 可选重载
                System.out.println("onRobotOnline => agentMeta " + agentMeta.toString());
            }

            @Override
            public void onRobotOffline(String agentId) {

            }

            /**
             * @param agentId    智能体Id（如AGENT_0000001）
             * @param eventId    会话id，一个查询一个唯一Id（如 event_r4aT53w0zKyMGJ4c3dgOS）
             * @param text       查询文本，一般为final asr
             * @param response   应答，二开开发者通过该对象接口返回asr识别结果及llm文本结果
             */
            @Override
            public void onRequest(
                    String agentId, String eventId, String text, Asr2TtsResponse response)
{

                // 可选获取自定义智能体默认元数据及自定义元数据
                AgentMeta agentMeta = agentSdk.getAgentMeta(agentId);
                // 智能体默认配置可直接通过函数名获取
                String defWakeupWord = agentMeta.getWakeupWord();
                String defCity = agentMeta.getCity();
                String defRobotName = agentMeta.getRobotName();
                String defBuzzWord = agentMeta.getBuzzWord();
                // 智能体自定义配置可通过参数key获取
                String yourParamKey = agentMeta.getString("您的自定义参数key");

                // TODO text（如"今天天气怎么样"为例）做语义理解（NLU），基于语义理解结果去做知识库查
询、工具调用等，再拿知识库查询结果或工具调用结果调用LLM做总结

                // LLM结果可能是多意图，需要给每个意图生成一个唯一的itemId，假设如下
                String itemId = "itemId_000001";
                String textDelta = "假设这就是LLM流式生成的结果";
                // 当LLM服务生成（某个意图）流式文本delta，则通过response应答（某个意图）流式文本
delta
                response.onLlmItemDelta(eventId, itemId, textDelta);
                // 当LLM服务生成（某个意图）结束，则通过response应答（某个意图）结束状态
                response.onLlmItemDone(eventId, itemId);
                // 当LLM服务生成（所有意图）结束，则通过response应答（所有意图）结束状态
                response.onLlmDone(eventId);

                // TODO 拿textDelta做TTS合成
                byte[] audio = new byte[1024]; // 假设这是tts服务合成的音频audio

                // 当TTS服务生成itemId的流式tts音频delta，则通过response应答流式tts音频delta
```

```
                response.onTtsItemDelta(eventId, itemId, audio);
                // 当TTS服务生成itemId的流式tts音频结束，则通过response应答流式tts音频结束状态
                response.onTtsItemDone(eventId, itemId);
                // 当TTS服务生成全部结束，则通过response应答全乎结束状态
                response.onTtsDone(eventId);

                // 当异常时，通过response返回异常信息
                /*
                int errorCode = 1010;
                String errorMsg = "假设调用asr出错了";
                response.onError(eventId, errorCode, errorMsg);
                */
            }
        });

        // 第四步：初始化
        agentSdk.initialize();
    }

}
```

5 AsrVideo2Vlm 接口开发示例

这里写描述

```
import com.agibot.aiem.sdk.AgentMeta;
import com.agibot.aiem.sdk.AgentSdk;
import com.agibot.aiem.sdk.passive.*;

public class AsrVideo2VlmExample {

    public static void main(String[] args) {

        String url = "wss://agentsdk.test.agibot.com/v1/agentsdk";
        String appId = "您通过灵心平台创建的二开智能体模版获取的appId";
        String appSecret = "您通过灵心平台创建的二开智能体模版获取的appSecret";

        // 第一步：创建sdk实例
        AgentSdk agentSdk = AgentSdk.create(url, appId, appSecret);

        // 第二步：注册验签回调接口
        agentSdk.registerAuth(new AgentAuthCallback() {
            @Override
            public void onAuthState(String appId, int code, String msg) {
                System.out.println("onAuthState => appId: " + appId + "code: " + code + ",
msg: " + msg);
            }
        });

        // 第三步：注册语义回调接口
        // 注册asrVideo2Vlm语义接口
```

```
        agentSdk.registerAsrVideo2Vlm(new AsrVideo2VlmCallback(agentSdk) {

            // TODO 因该实例为单例，非线程安全的资源对象可基于eventId做隔离，并在用完后及时销毁

            @Override
            public void onRobotOnline(String agentId, AgentMeta agentMeta) {
                // 可选重载
                System.out.println("onRobotOnline => agentMeta " + agentMeta.toString());
            }

            @Override
            public void onRobotOffline(String agentId) {

            }

            /**
             * @param agentId    智能体Id（如AGENT_0000001）
             * @param eventId    会话id，一个查询一个唯一Id（如 event_r4aT53w0zKyMGJ4c3dgOS）
             * @param flag       3 文本，4 h264视频流，5 jpg图片
             * @param text       查询文本，一般为final asr识别结果（当flag为3时有效）
             * @param buf        h264视频流（当flag为4时有效），jpg图片（当flag为5时有效）
             * @param response   应答，二开开发者通过该对象接口返回asr识别结果及llm文本结果
             */
            @Override
            public void onRequest(
                    String agentId, String eventId,
                    int flag, String text, byte[] buf, AsrVideo2VlmResponse response) {

                // 可选获取自定义智能体默认元数据及自定义元数据
                AgentMeta agentMeta = agentSdk.getAgentMeta(agentId);
                // 智能体默认配置可直接通过函数名获取
                String defWakeupWord = agentMeta.getWakeupWord();
                String defCity = agentMeta.getCity();
                String defRobotName = agentMeta.getRobotName();
                String defBuzzWord = agentMeta.getBuzzWord();
                // 智能体自定义配置可通过参数key获取
                String yourParamKey = agentMeta.getString("您的自定义参数key");

                // TODO 拿text（如"今天天气怎么样"为例）做语义理解（NLU），基于语义理解结果去做知识库查询、工具调用等，再拿知识库查询结果或工具调用等结果调用LLM做总结

                // LLM结果可能是多意图，需要给每个意图生成一个唯一的itemId，假设如下
                String itemId = "itemId_000001";
                String textDelta = "假设这就是LLM流式生成的结果";
                // 当LLM服务生成（某个意图）流式文本delta，则通过response应答（某个意图）流式文本delta
                response.onVlmItemDelta(eventId, itemId, textDelta);
                // 当LLM服务生成（某个意图）结束，则通过response应答（某个意图）结束状态
                response.onVlmItemDone(eventId, itemId);
                // 当LLM服务生成（所有意图）结束，则通过response应答（所有意图）结束状态
                response.onVlmDone(eventId);
```

```
                // 当异常时，通过response返回异常信息
                /*
                int errorCode = 1010;
                String errorMsg = "假设调用asr出错了";
                response.onError(eventId, errorCode, errorMsg);
                */
            }
        });

        // 第四步：初始化
        agentSdk.initialize();
    }

}
```

6 AsrVideo2Tts 接口开发示例

这里写描述

```
import com.agibot.aiem.sdk.AgentMeta;
import com.agibot.aiem.sdk.AgentSdk;
import com.agibot.aiem.sdk.passive.*;

public class AsrVideo2TtsExample {

    public static void main(String[] args) {

        String url = "wss://agentsdk.test.agibot.com/v1/agentsdk";
        String appId = "您通过灵心平台创建的二开智能体模版获取的appId";
        String appSecret = "您通过灵心平台创建的二开智能体模版获取的appSecret";

        // 第一步：创建sdk实例
        AgentSdk agentSdk = AgentSdk.create(url, appId, appSecret);

        // 第二步：注册验签回调接口
        agentSdk.registerAuth(new AgentAuthCallback() {
            @Override
            public void onAuthState(String appId, int code, String msg) {
                System.out.println("onAuthState => appId: " + appId + "code: " + code + ",
msg: " + msg);
            }
        });

        // 第三步：注册语义回调接口
        // 注册asrVideo2tts语义接口
        agentSdk.registerAsrVideo2Tts(new AsrVideo2TtsCallback(agentSdk) {

            // TODO 因该实例为单例，非线程安全的资源对象可基于eventId做隔离，并在用完后及时销毁

            @Override
            public void onRobotOnline(String agentId, AgentMeta agentMeta) {
```

```
                // 可选重载
                System.out.println("onRobotOnline => agentMeta " + agentMeta.toString());
            }

            @Override
            public void onRobotOffline(String agentId) {

            }

            /**
             * @param agentId    智能体Id（如AGENT_0000001）
             * @param eventId    会话id，一个查询一个唯一Id（如 event_r4aT53w0zKyMGJ4c3dgOS）
             * @param flag       3 h264视频流
             * @param text       查询文本，一般为final asr
             * @param buf        h264视频流（当flag为3时有效）
             * @param response   应答，二开开发者通过该对象接口返回asr识别结果及llm文本结果
             */
            @Override
            public void onRequest(
                    String agentId, String eventId, int flag, String text, byte[] buf,
AsrVideo2TtsResponse response) {

                // 可选获取自定义智能体默认元数据及自定义元数据
                AgentMeta agentMeta = agentSdk.getAgentMeta(agentId);
                // 智能体默认配置可直接通过函数名获取
                String defWakeupWord = agentMeta.getWakeupWord();
                String defCity = agentMeta.getCity();
                String defRobotName = agentMeta.getRobotName();
                String defBuzzWord = agentMeta.getBuzzWord();
                // 智能体自定义配置可通过参数key获取
                String yourParamKey = agentMeta.getString("您的自定义参数key");

                // TODO text（如"今天天气怎么样"为例）做语义理解（NLU），基于语义理解结果去做知识库查
询、工具调用等，再拿知识库查询结果或工具调用结果调用LLM做总结

                // LLM结果可能是多意图，需要给每个意图生成一个唯一的itemId，假设如下
                String itemId = "itemId_000001";
                String textDelta = "假设这就是LLM流式生成的结果";
                // 当LLM服务生成（某个意图）流式文本delta，则通过response应答（某个意图）流式文本
delta
                response.onVlmItemDelta(eventId, itemId, textDelta);
                // 当LLM服务生成（某个意图）结束，则通过response应答（某个意图）结束状态
                response.onVlmItemDone(eventId, itemId);
                // 当LLM服务生成（所有意图）结束，则通过response应答（所有意图）结束状态
                response.onVlmDone(eventId);

                // TODO 拿textDelta做TTS合成
                byte[] audio = new byte[1024]; // 假设这是tts服务合成的音频audio

                // 当TTS服务生成itemId的流式tts音频delta，则通过response应答流式tts音频delta
                response.onTtsItemDelta(eventId, itemId, audio);
                // 当TTS服务生成itemId的流式tts音频结束，则通过response应答流式tts音频结束状态
```

```
                response.onTtsItemDone(eventId, itemId);
                // 当TTS服务生成全部结束，则通过response应答全乎结束状态
                response.onTtsDone(eventId);

                // 当异常时，通过response返回异常信息
                /*
                int errorCode = 1010;
                String errorMsg = "假设调用asr出错了";
                response.onError(eventId, errorCode, errorMsg);
                */
            }
        });

        // 第四步：初始化
        agentSdk.initialize();
    }

}
```

7 AudioVideo2Vlm 接口开发示例

这里写描述

```
import com.agibot.aiem.sdk.AgentMeta;
import com.agibot.aiem.sdk.AgentSdk;
import com.agibot.aiem.sdk.passive.*;

public class AudioVideo2VlmExample {

    public static void main(String[] args) {

        String url = "wss://agentsdk.test.agibot.com/v1/agentsdk";
        String appId = "您通过灵心平台创建的二开智能体模版获取的appId";
        String appSecret = "您通过灵心平台创建的二开智能体模版获取的appSecret";

        // 第一步：创建sdk实例
        AgentSdk agentSdk = AgentSdk.create(url, appId, appSecret);

        // 第二步：注册验签回调接口
        agentSdk.registerAuth(new AgentAuthCallback() {
            @Override
            public void onAuthState(String appId, int code, String msg) {
                System.out.println("onAuthState => appId: " + appId + "code: " + code + ",
msg: " + msg);
            }
        });

        // 第三步：注册语义回调接口
        // 注册audio2llm语义接口
        agentSdk.registerAudioVideo2Vlm(new AudioVideo2VlmCallback(agentSdk) {
```

```
            // TODO 因该实例为单例，非线程安全的资源对象可基于eventId做隔离，并在用完后及时销毁

            @Override
            public void onRobotOnline(String agentId, AgentMeta agentMeta) {
                // 可选重载
                System.out.println("onRobotOnline => agentMeta " + agentMeta.toString());
            }

            @Override
            public void onRobotOffline(String agentId) {

            }

            /**
             * @param agentId    智能体Id（如AGENT_0000001）
             * @param eventId    会话id，一个查询一个唯一Id（如 event_r4aT53w0zKyMGJ4c3dgOS）
             * @param flag       0 VAD开始，1中间音频，2 VAD结束，3 h264视频流
             * @param buf        音频字节数组（当flag为1时有效），h264视频流（当flag为3时有效）
             * @param response   应答，二开开发者通过该对象接口返回asr识别结果及llm文本结果
             */
            @Override
            public void onRequest(
                    String agentId, String eventId, int flag, byte[] buf,
AudioVideo2VlmResponse response) {

                // 可选获取自定义智能体默认元数据及自定义元数据
                AgentMeta agentMeta = agentSdk.getAgentMeta(agentId);
                // 智能体默认配置可直接通过函数名获取
                String defWakeupWord = agentMeta.getWakeupWord();
                String defCity = agentMeta.getCity();
                String defRobotName = agentMeta.getRobotName();
                String defBuzzWord = agentMeta.getBuzzWord();
                // 智能体自定义配置可通过参数key获取
                String yourParamKey = agentMeta.getString("您的自定义参数key");

                // TODO 调用ASR逻辑
                if (flag == 0) {
                    // 如初始化asr的客户端
                } else if (flag == 1) {
                    // 给asr传音频
                } else if (flag == 2) {
                    //
                }
                String asrMiddle = "今天"; // 假设这是一个asr的中间识别结果。
                String asrFinal = "今天天气咋样"; // 假设这是一个asr的中间识别结果。

                // 当ASR服务返回流式中间结果，则通过response应答中间结果
                response.onAsrMiddle(eventId, asrMiddle);
                // 当ASR服务返回最终结果，则通过response应答最终结果
                response.onAsrFinal(eventId, asrFinal);
```

```
                // TODO 拿asrFinal（以"今天天气怎么样"为例）做语义理解（NLU），基于语义理解结果去做知识库查询、工具调用等，再拿知识库查询结果或工具调用等结果调用LLM做总结

                // LLM结果可能是多意图，需要给每个意图生成一个唯一的itemId，假设如下
                String itemId = "itemId_000001";
                String textDelta = "假设这就是LLM流式生成的结果";
                // 当LLM服务生成（某个意图）流式文本delta，则通过response应答（某个意图）流式文本delta
                response.onVlmItemDelta(eventId, itemId, textDelta);
                // 当LLM服务生成（某个意图）结束，则通过response应答（某个意图）结束状态
                response.onVlmItemDone(eventId, itemId);
                // 当LLM服务生成（所有意图）结束，则通过response应答（所有意图）结束状态
                response.onVlmDone(eventId);

                // 当异常时，通过response返回异常信息
                /*
                int errorCode = 1010;
                String errorMsg = "假设调用asr出错了";
                response.onError(eventId, errorCode, errorMsg);
                */
            }
        });

        // 第四步：初始化
        agentSdk.initialize();
    }

}
```

8 AudioVideo2Tts 接口开发示例

这里写描述

```
import com.agibot.aiem.sdk.AgentMeta;
import com.agibot.aiem.sdk.AgentSdk;
import com.agibot.aiem.sdk.passive.*;

public class AudioVideo2TtsExample {

    public static void main(String[] args) {

        String url = "wss://agentsdk.test.agibot.com/v1/agentsdk";
        String appId = "您通过灵心平台创建的二开智能体模版获取的appId";
        String appSecret = "您通过灵心平台创建的二开智能体模版获取的appSecret";

        // 第一步：创建sdk实例
        AgentSdk agentSdk = AgentSdk.create(url, appId, appSecret);

        // 第二步：注册验签回调接口
        agentSdk.registerAuth(new AgentAuthCallback() {
            @Override
```

```
            public void onAuthState(String appId, int code, String msg) {
                System.out.println("onAuthState => appId: " + appId + "code: " + code + ",
msg: " + msg);
            }
        });

        // 第三步：注册语义回调接口
        // 注册audioVideo2llm语义接口
        agentSdk.registerAudioVideo2Tts(new AudioVideo2TtsCallback(agentSdk) {

            // TODO 因该实例为单例，非线程安全的资源对象可基于eventId做隔离，并在用完后及时销毁

            @Override
            public void onRobotOnline(String agentId, AgentMeta agentMeta) {
                // 可选重载
                System.out.println("onRobotOnline => agentMeta " + agentMeta.toString());
            }

            @Override
            public void onRobotOffline(String agentId) {

            }

            /**
             * @param agentId    智能体Id（如AGENT_0000001）
             * @param eventId    会话id，一个查询一个唯一Id（如 event_r4aT53w0zKyMGJ4c3dgOS）
             * @param flag       0 VAD开始，1中间音频，2 VAD结束，3 h264视频流
             * @param buf        音频字节数组（当flag为1时有效），h264视频流（当flag为3时有效）
             * @param response   应答，二开开发者通过该对象接口返回asr识别结果及llm文本结果
             */
            @Override
            public void onRequest(
                    String agentId, String eventId, int flag, byte[] buf,
AudioVideo2TtsResponse response) {

                // 可选获取自定义智能体默认元数据及自定义元数据
                AgentMeta agentMeta = agentSdk.getAgentMeta(agentId);
                // 智能体默认配置可直接通过函数名获取
                String defWakeupWord = agentMeta.getWakeupWord();
                String defCity = agentMeta.getCity();
                String defRobotName = agentMeta.getRobotName();
                String defBuzzWord = agentMeta.getBuzzWord();
                // 智能体自定义配置可通过参数key获取
                String yourParamKey = agentMeta.getString("您的自定义参数key");


                // TODO 调用ASR逻辑
                if (flag == 0) {
                    // 如初始化asr的客户端
                } else if (flag == 1) {
                    // 给asr传音频
                } else if (flag == 2) {
```

```
                    //
                }
                String asrMiddle = "今天"; // 假设这是一个asr的中间识别结果。
                String asrFinal = "今天天气咋样"; // 假设这是一个asr的中间识别结果。

                // 当ASR服务返回流式中间结果，则通过response应答中间结果
                response.onAsrMiddle(eventId, asrMiddle);
                // 当ASR服务返回最终结果，则通过response应答最终结果
                response.onAsrFinal(eventId, asrFinal);

                // TODO 拿asrFinal（以"今天天气怎么样"为例）做语义理解（NLU），基于语义理解结果去做知识库查询、工具调用等，再拿知识库查询结果或工具调用结果调用LLM做总结

                // LLM结果可能是多意图，需要给每个意图生成一个唯一的itemId，假设如下
                String itemId = "itemId_000001";
                String textDelta = "假设这就是LLM流式生成的结果";
                // 当LLM服务生成（某个意图）流式文本delta，则通过response应答（某个意图）流式文本delta
                response.onVlmItemDelta(eventId, itemId, textDelta);
                // 当LLM服务生成（某个意图）结束，则通过response应答（某个意图）结束状态
                response.onVlmItemDone(eventId, itemId);
                // 当LLM服务生成（所有意图）结束，则通过response应答（所有意图）结束状态
                response.onVlmDone(eventId);

                // TODO 拿textDelta做TTS合成
                byte[] audio = new byte[1024]; // 假设这是tts服务合成的音频audio

                // 当TTS服务生成itemId的流式tts音频delta，则通过response应答流式tts音频delta
                response.onTtsItemDelta(eventId, itemId, audio);
                // 当TTS服务生成itemId的流式tts音频结束，则通过response应答流式tts音频结束状态
                response.onTtsItemDone(eventId, itemId);
                // 当TTS服务生成全部结束，则通过response应答全乎结束状态
                response.onTtsDone(eventId);

                // 当异常时，通过response返回异常信息
                /*
                int errorCode = 1010;
                String errorMsg = "假设调用asr出错了";
                response.onError(eventId, errorCode, errorMsg);
                */
            }
        });

        // 第四步：初始化
        agentSdk.initialize();
    }

}
```

主动交互开发

待建设...

拉流开发

待建设...

Python 语言SDK

待建设...